

Prague University of Economics and Business
Faculty of Informatics and Statistics



Self-Corrective Assistant for Software Development

BACHELOR THESIS

Study program: Data Analytics

Author: Maksim Obukhov

Supervisor: Ing. Filip Vencovský, Ph.D.

Prague, December 2024

Acknowledgements

I would like to sincerely thank Ing. Filip Vencovský, Ph.D., for his invaluable support throughout my thesis journey. His teaching has not only enhanced my understanding of the subject but also ignited my passion for software engineering and natural language processing. His guidance and expertise have played a crucial role in shaping this research.

Abstract

Programming plays a central role in modern problem-solving. The advent of Large Language Models has significantly advanced program synthesis, facilitating automatic code generation. However, LLMs still struggle with complex algorithmic problems required developer’s thinking and precision in implementation. This study introduces a self-corrective multi-agent system designed to enhance the problem-solving abilities of LLMs. The system employs a graph-based workflow with seven specialized agents: Simplifier, Self-retrieval, Planning, Ranking, Test Case Generator, Coding, and an Iterative Self-correction agent, along with a code execution node. These agents work together to break down problems, retrieve relevant data, generate and evaluate multiple solutions, test the generated code, and refine it iteratively. The system’s performance is evaluated using DeepMind’s CodeContests dataset, benchmarking it against established models like Reflexion, AlphaCodium, and MapCoder. The proposed system outperforms existing baselines, achieving a Pass@1 rate of 26.50% and a Pass@4 rate of 35.04%. In terms of cost-effectiveness, it demonstrates an average cost per problem of \$0.0883 and an average token usage of 17,082.46 tokens per solution, outperforming state-of-the-art models in both efficacy and efficiency.

Keywords

large language models, code generation, algorithmic problem-solving, multi-agent system, competitive programming

Contents

1	Introduction	6
1.1	Context and motivation	6
1.2	Problem statement	6
1.3	Research objectives	7
2	Background and Related Work	8
2.1	Prompting techniques	8
2.1.1	Self-attention mechanism	8
2.1.2	Basic prompting techniques	8
2.1.3	Advanced prompting techniques	9
2.2	LLMs in code generation tasks	10
2.2.1	Early approaches	10
2.2.2	Open-source and Specialised coding models	10
2.3	Multi-agent systems in code generation tasks	11
2.3.1	Architectural overview	11
2.3.2	Notable implementations	11
3	System architecture	13
3.1	Architecture overview	13
3.1.1	Proposed architecture	13
3.1.2	Objective	13
3.2	Agents	14
3.2.1	Problem simplification agent	14
3.2.2	Self-retrieval agent	16
3.2.3	Planning agent	17
3.2.4	Ranking agent	17
3.2.5	Test generation agent	18
3.2.6	Coding agent	18
3.2.7	Test evaluation node	19
3.2.8	Debugging Agent	20
4	Technical details	22
4.1	Frameworks overview	22
4.2	Graph-based workflow orchestration	22
4.2.1	Workflow representation	22
4.2.2	State management	22
5	Experimental evaluation	24
5.1	Experimental setup	24
5.1.1	Dataset preprocessing	24

5.1.2	Language model utilised	24
5.2	Evaluation metrics	24
5.2.1	Solution quality metrics	25
5.2.2	Efficiency metrics	25
5.2.3	Resource utilisation metrics	25
5.2.4	Testing coverage metrics	25
6	Results and discussion	26
6.1	Key findings	26
6.1.1	Pass rate distribution	26
6.1.2	Comparison with Baselines	26
6.1.3	Difficulty-stratified analysis	27
6.1.4	Impact of self-correction mechanism	28
6.2	Testing and validation metrics	28
6.3	Resource utilisation analysis	29
6.3.1	LLM interaction patterns	29
6.3.2	Computational efficiency	29
6.3.3	Comparison of calls for Solved problems with Baselines	29
6.4	Key patterns and implications	29
6.5	Limitations and bottlenecks	30
6.6	Future work	30
6.6.1	Performance improvements	31
6.6.2	Implement long-term memory	31
6.6.3	Adaptive agent specialisation	31
7	Conclusions	32
	List of references	33

1. Introduction

1.1 Context and motivation

Programming has become an essential problem-solving instrument that impacts all aspects of modern life (Knuth, 1992; Y. Li et al., 2022; Parvez et al., 2018). The advent of Large Language Models (LLMs) has revolutionized program synthesis, enabling automatic code generation with minimal human oversight (Chowdhery et al., 2022; Nijkamp et al., 2022). However, even as models grow in size and training data expands, they continue to face challenges with complex problem-solving tasks (Austin et al., 2021).

Professional developers regularly encounter sophisticated algorithmic challenges reminiscent of competitive programming problems. These challenges are critical in scenarios requiring advanced algorithmic thinking, particularly for:

- Critical system component optimization
- Complex data pipeline implementation
- Scale-efficient algorithm design
- Technical bottleneck resolution
- Management of computation-intensive operations

1.2 Problem statement

Contemporary LLM-based code generation typically relies on direct prompting, generating code from problem specifications and input/output examples (Chen et al., 2021). While newer techniques such as chain-of-thought (CoT) promote modular generation and planning (Wei et al., 2022), and retrieval-based methods utilize similar problems for guidance, these approaches show limited effectiveness when tackling complex algorithmic problems, often failing comprehensive testing (Ridnik et al., 2024). The primary limitations stem from three key challenges:

1. Insufficient problem decomposition in complex scenarios
2. Lack of systematic planning before implementation
3. Inability to run and test the generated code, limiting effective debugging and correction

These challenges are particularly evident in competitive programming problems, where solutions require both algorithmic insight and precise implementation.

1.3 Research objectives

This study aims to address the aforementioned limitations by introducing a self-corrective multi-agent system designed to enhance the problem-solving capabilities of LLMs through a graph-based workflow, combining the strengths of previous approaches while mitigating their weaknesses. The system employs 7 specialized agents and 1 additional node:

- **Simplifier agent:** Deconstructs the essence of a given problem.
- **Self-retrieval agent:** Leverages similar problems for guidance.
- **Planning agent:** Generates T multiple solution approaches.
- **Ranking agent:** Evaluates proposed solutions before implementation.
- **Test case generator agent:** Creates test cases before generating code.
- **Coding agent:** Implements the evaluated solutions.
- **Code execution node:** Runs generated code on test cases.
- **Iterative self-correction agent:** Attempts up to K debugging cycles to refine given solution.

For evaluation, the study employs DeepMind’s CodeContests dataset of competitive programming problems. The proposed system outperforms strong baselines such as Reflexion (Shinn et al., 2023) and AlphaCodium (Ridnik et al., 2024), demonstrating superior cost-effectiveness and efficacy compared to current state-of-the-art architectures like MapCoder (Md. Ashraful et al., 2024). The results include a Pass@1 rate of 26.50% and a Pass@4 rate of 35.04%, with an average cost per problem of \$0.0883 and an average token usage of 17,082.46 tokens per solution.

2. Background and Related Work

2.1 Prompting techniques

This section provides theoretical foundations and examines relevant research in prompting techniques for Large Language Models (LLMs), progressing from fundamental mechanisms to advanced methodologies.

2.1.1 Self-attention mechanism

The self-attention mechanism, introduced by (Vaswani et al., 2017) in the Transformer architecture, forms the foundational building block of modern LLMs. Such a mechanism enables models to process input sequences by dynamically weighting the importance of different tokens in relation to each other. In the context of prompting, self-attention plays a crucial role in how models interpret and respond to prompts by:

- Establishing long-range dependencies between tokens in the input sequence
- Creating contextual representations that capture the relationships between different parts of the prompt
- Enabling the model to focus on relevant information while processing sequential data

The self-attention computation can be formally expressed as:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V,$$

where Q (Query), K (Key), and V (Value) are learned transformations of the input, and d_k is the dimension of the key vectors. The mechanism's ability to capture complex relationships between tokens laid the groundwork for more sophisticated prompting techniques.

2.1.2 Basic prompting techniques

Zero-shot prompting

Zero-shot prompting represents the most straightforward approach to model interaction, where tasks are performed without any exemplars. This technique relies heavily on the model's pre-trained knowledge and the self-attention mechanism's ability to comprehend task specifications directly. For instance, in code generation tasks, zero-shot prompting involves providing a problem description and expecting the model to generate appropriate code without examples.

Few-shot prompting

Few-shot prompting enhances zero-shot approaches by including examples within the prompt. This technique leverages the self-attention mechanism to establish patterns between examples and the target task. The model’s attention layers can identify relevant patterns across the provided examples and apply them to new situations.

2.1.3 Advanced prompting techniques

Chain-of-Thought

Chain-of-Thought (CoT) prompting, introduced by (Wei et al., 2022), represents a significant advancement in prompting techniques. This approach builds upon the self-attention mechanism’s capability to process sequential reasoning steps. The connection between self-attention and CoT is particularly notable:

- The self-attention mechanism’s ability to maintain context across long sequences enables CoT to develop extended reasoning chains
- Each step in the chain receives focused attention, allowing the model to build coherent logical progressions
- Self-attention helps establish relationships between different reasoning steps, creating a more robust problem-solving process

CoT has demonstrated particular effectiveness in complex programming tasks, where step-by-step reasoning is crucial for (a) problem decomposition, (b) algorithm design, (c) implementation planning, (d) error identification.

ReAct

ReAct (Reasoning and Acting) extends CoT by incorporating action steps alongside reasoning. This framework, proposed by (Yao et al., 2023), interleaves thought processes with concrete actions. In programming contexts, ReAct enables models to:

- Reason about potential solutions
- Take actions such as writing code snippets
- Observe the results
- Adjust subsequent steps based on observations

2.2 LLMs in code generation tasks

2.2.1 Early approaches

Program synthesis has evolved significantly from its early rule-based approaches (Gulwani et al., 2017) to modern neural methods. The initial breakthrough came with sequence-to-sequence models (Sutskever et al., 2014), which demonstrated the first practical results in translating natural language to code. This advancement was further enhanced by the introduction of the Transformer architecture (Vaswani et al., 2017), which revolutionized code generation through its ability to capture long-range dependencies and contextual information in programming languages.

Early neural approaches to code generation primarily focused on fine-tuning techniques. (Wang et al., 2021) and (Ahmad et al., 2021) demonstrated the effectiveness of specialized training objectives for code generation, while (Feng et al., 2020) explored the integration of programming language syntax into the training process. These efforts laid the foundation for more sophisticated models but were constrained by the limited size and capabilities of the underlying architectures.

The landscape of code generation underwent a dramatic shift with the emergence of large language models specifically designed for programming tasks. (Y. Li et al., 2022) and (Austin et al., 2021) introduced some of the first large-scale models trained explicitly on code, showcasing significant improvements over previous approaches. (Nijkamp et al., 2022) further advanced this field by introducing specialized pre-training objectives tailored for code understanding and generation, though their work highlighted persistent challenges in handling complex algorithmic problems.

2.2.2 Open-source and Specialised coding models

Recent developments in open-source LLMs have shown promising results in code generation. The Llama-2 model (Touvron et al., 2023) demonstrated strong general-purpose coding capabilities, particularly in zero-shot scenarios. Building upon the Llama-2 foundation, CodeLlama-2 (Roziere et al., 2023) introduced specialized modifications for code generation, including an extended context window and an enhanced token vocabulary tailored for programming languages. Additionally, models like Deepseek Coder (Guo et al., 2024) and MoTCoder (J. Li et al., 2023) represent advancements in handling complex programming tasks. Deepseek Coder focuses on improving algorithmic reasoning and solution optimization, while MoTCoder emphasizes modular thinking in code generation, demonstrating enhanced capabilities in problem decomposition. However, both models still struggle with consistent performance in complex problem-solving scenarios, particularly in maintaining logical consistency across lengthy solutions.

2.3 Multi-agent systems in code generation tasks

Recent advances in code generation have witnessed a significant shift towards multi-agent architectures, building upon the foundation of LLMs. This paradigm draws inspiration from human software development practices, where complex programming challenges are addressed through collaborative effort.

2.3.1 Architectural overview

Multi-agent systems for code generation typically employ a distributed architecture where specialized agents handle distinct aspects of the development process. Such a modular approach enables scalable solutions through the strategic deployment of agents focused on specific tasks such as code synthesis, error detection, and documentation generation. The effectiveness of these systems is further enhanced by chain-of-thought mechanisms, which allow agents to decompose complex problems into manageable sub-tasks while maintaining logical consistency throughout the development lifecycle.

2.3.2 Notable implementations

The progression of multi-agent frameworks for code generation has been driven by several notable advancements. AlphaCode, developed by DeepMind in 2022, introduced an innovative approach to competitive programming by generating a vast array of candidate solutions and refining them through clustering and filtering processes. This strategy allowed AlphaCode to produce high-quality code for challenging problems, establishing its potential in solving complex programming tasks effectively.

Reflexion, proposed by Shinn et al. in 2023, further advanced the field by enhancing code generation through iterative refinement. By enabling large language models to evaluate and improve their outputs based on feedback from test cases, Reflexion demonstrated a significant improvement in problem-solving accuracy. This iterative feedback loop allowed for more reliable and refined code outputs, highlighting the power of adaptive learning techniques in programming.

Building on these innovations, AlphaCodium, introduced by Ridnik et al. in 2024, presented a test-driven, multi-stage approach to code generation. By incorporating iterative testing and refinement phases, AlphaCodium significantly enhanced the performance of large language models on complex programming tasks. This methodology demonstrated the effectiveness of combining rigorous evaluation with structured refinement in achieving superior accuracy on coding benchmarks.

MapCoder, proposed by Md. Ashraful et al. in 2024, represents a leap forward in multi-agent architectures for code generation. By emulating the human programming cycle – retrieving

relevant examples, planning, coding, and debugging – MapCoder achieves enhanced capabilities for competitive problem-solving. This human-like iterative process allows MapCoder to deliver state-of-the-art performance across various competitive programming benchmarks, further establishing the role of multi-agent frameworks in tackling sophisticated coding challenges.

These developments collectively illustrate the benefits of multi-agent architectures over traditional single-agent models in code generation. By employing specialised agents and integrating advanced coordination mechanisms, these systems demonstrate superior precision and efficiency in addressing complex programming challenges, thereby setting a new standard in the field.

3. System architecture

3.1 Architecture overview

3.1.1 Proposed architecture

The proposed architecture introduces a novel multi-agent system for code generation, emphasising efficiency and adaptability through its undirected cyclic graph structure. This design diverges from the linear pipelines of AlphaCode by enabling dynamic interactions between agents, allowing for iterative refinement without the rigid sequential workflows of the previous system.

A key innovation lies in the integration of two specialized agents: the *Problem Simplification Agent* and the *Test Generation Agent*. The *Problem Simplification Agent* systematically decomposes complex problems into XML-formatted components, providing a clear and structured foundation for downstream agents. This contrasts with any other system ad-hoc problem interpretation and enhances clarity and efficiency at the start of the pipeline. The *Test Generation Agent* creates robust and diverse test cases tailored to validate edge cases and requirements, a capability not present in Reflexion and MapCoder.

Another significant advancement is the architecture’s approach to solution planning and evaluation. While MapCoder iterates over multiple plans T to refine and validate each one – leading to substantial token, time, and computational costs – this architecture generates T plans, ranks them using a *Ranking Agent*, and selects only the highest-scoring plan for further processing. This selective approach drastically reduces resource consumption while maintaining high solution quality, offering a clear advantage in cost-efficiency compared to MapCoder.

By combining these innovations with its modular and scalable design, the proposed architecture addresses the limitations of existing systems while introducing a more structured, efficient, and computationally optimised solution.

3.1.2 Objective

The objective is to create a novel multi-agent system tailored for contest-level programming challenges, specifically designed to address the strengths and weaknesses of existing solutions discussed above. This system will consist of 7 specialised LLM agents and a dedicated test evaluation node:

1. Problem simplification agent
2. Self-retrieval agent

3. Planning agent
4. Ranking agent
5. Test generation agent
6. Coding agent
7. Test evaluation node
8. Debugger agent

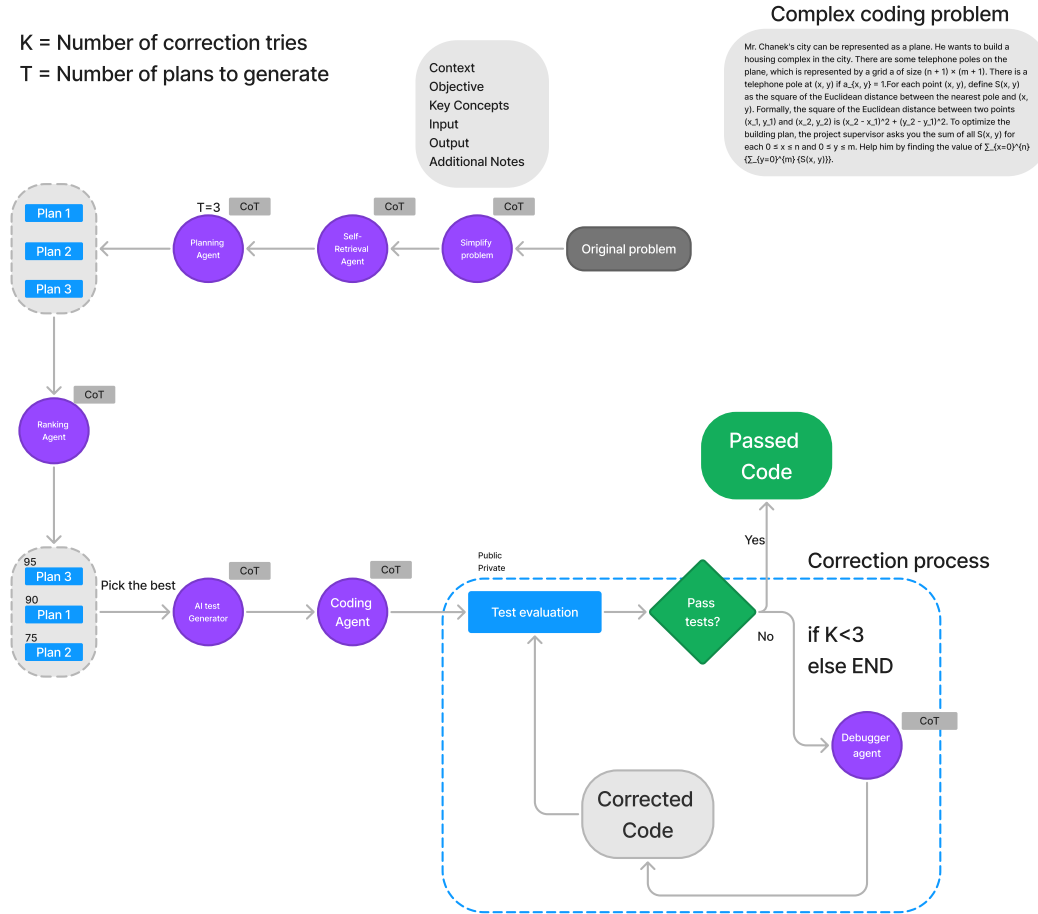


Figure 3.1: System architecture of the proposed multi-agent system

3.2 Agents

3.2.1 Problem simplification agent

Operational framework

The *Problem Simplification Agent* operates based on a meticulously crafted system prompt (Appendix A) that defines its core functionality and operational guidelines. The prompt

instructs the agent to assume the role of a world-class competitive programming problem solver, emphasizing the importance of delivering a clear and comprehensive description of a given complex coding contest problem. The agent is explicitly directed to focus on key aspects such as problem context, objectives, inputs, outputs, rules, constraints, and other pertinent details.

Analytical processing and Structured output

A critical component of the system prompt is the incorporation of a *chain-of-thought* process, encapsulated within `<thinking>` tags preceding each corresponding XML section. This approach ensures that the agent engages in a deliberate and systematic analysis of each problem aspect before generating the structured output. The culmination of such analysis is the production of a well-structured XML-formatted document that adheres to a predefined schema. This schema organizes the problem description into distinct sections, each encapsulated within specific XML tags such as `<title>`, `<context>`, `<objective>`, `<inputs>`, `<outputs>`, `<rules>`, `<constraints>`, and `<relevant_details>`.

The use of XML formatting facilitates structured data representation, which is conducive to further processing, analysis, or integration with other systems. Language models like OpenAI's GPT-4 or Anthropic's Claude, specifically fine-tuned to understand such XML tags, can leverage this structured format to enhance their ability to process complex inputs and outputs more effectively.

Component-wise Breakdown

1. **Title (`<title>`):** Captures the essence of the problem through a concise and descriptive title, providing an immediate overview of the problem's focus.
2. **Context (`<context>`):** Delineates the background and setting of the problem, elucidating the scenario or abstract environment in which the problem is situated. This section includes bullet points that highlight the real-world or theoretical underpinnings relevant to the problem.
3. **Objective (`<objective>`):** Clearly articulates the primary goal and any secondary objectives inherent in the problem. This segment ensures that the purpose and desired outcomes of the problem are unambiguously understood.
4. **Inputs (`<inputs>`):** Details the format, type, and constraints of the inputs required for the problem. By specifying the range and nature of inputs, this section aids in defining the parameters within which the solution must operate.
5. **Outputs (`<outputs>`):** Specifies the expected output format and any precision or formatting requirements. This ensures that the solution aligns with the problem's expectations regarding result presentation.
6. **Rules (`<rules>`):** Enumerates both explicit and implicit rules that govern the problem. This includes any conditions, limitations, or special cases that must be adhered to,

thereby outlining the boundaries of permissible solutions.

7. **Constraints (<constraints>)**: Lists the limitations and restrictions imposed on the problem, such as time complexity, space complexity, or other resource-based constraints. Understanding these constraints is crucial for devising efficient and viable solutions.
8. **Relevant Details (<relevant_details>)**: Captures additional information that may influence the problem-solving approach. This includes clarifications, special conditions, or ancillary information that provides deeper insights into the problem’s nuances.

3.2.2 Self-retrieval agent

Operational framework

The core idea of the *Self-Retrieval Agent* is to leverage the language model’s inherent capability to recall T similar problems and corresponding solution plans from its training data, eliminating the need for a dedicated external database and additional retrieval logic. This agent operates through a carefully designed system prompt (Appendix A) that guides the model in identifying, analysing, and explaining relevant problems and their underlying algorithms.

Analytical processing and Structured output

The agent employs a systematic approach to problem analysis and solution generation, utilising XML formatting for structured output. Each retrieved problem undergoes a comprehensive analysis through multiple CoT stages, ensuring thorough consideration of various aspects and approaches. The output maintains consistency with the established XML schema while incorporating additional tags specific to algorithm identification and tutorial generation.

Component-wise breakdown

1. **Problem Description (<description>)**: Presents a comprehensive overview of the retrieved problem, establishing its relevance to the original problem.
2. **Solution Plan (<plan>)**: Outlines a detailed, step-by-step approach to solving the retrieved problem, presented as a numbered list.
3. **Implementation (<code>)**: Provides the actual implementation of the solution in the specified programming language.
4. **Algorithm Analysis (<algorithm>)**:
 - **Name (<name>)**: Identifies the specific algorithm category.
 - **Tutorial (<tutorial>)**: Provides a short explanation of the algorithm.
5. **Problem Set Generation**: Retrieves $T_2, T_3, \dots, T_i$ distinct problems, each following the same structured format while maintaining relevance to the original problem.

3.2.3 Planning agent

Operational framework

The *Planning Agent* focuses on generating T multiple distinct solution plans based on the simplified problem description provided by the *Simplification Agent*. It leverages a structured thinking process to analyze the problem requirements and develop T comprehensive solution plans, each utilizing different algorithmic approaches.

Analytical processing and Structured output

The agent employs a CoT process, examining the problem from multiple perspectives to generate diverse solution strategies. Each proposed solution undergoes thorough evaluation through structured thinking stages, with output formatted in XML to maintain consistency and clarity.

Component-wise breakdown

First Solution Strategy:

- **Algorithm Identification (<algorithm_name>):** The algorithm (e.g., Brute-force, Dynamic Programming, Divide-and-Conquer, Greedy, Backtracking, Recursive, Binary Search) that needs to be used to solve the given problem.
- **Detailed Implementation Plan (<plan>):** Concrete plan to implement the algorithm, presented as a numbered list.

Remaining Solution Strategies: Similarly creates $T_2, T_3, \dots, T_i$ distinct plans based on the previous implementation.

3.2.4 Ranking agent

Operational framework

The *Ranking Agent* independently evaluates each solution plan generated by the *Planning Agent*, providing a scoring mechanism to assess the viability, efficiency, and potential effectiveness of the proposed approach.

Scoring methodology

The agent implements a CoT process as a scoring system that objectively analyzes multiple dimensions of the given generated solution strategy, ensuring a rigorous and multi-faceted

evaluation process.

During the evaluation process, each solution plan undergoes an independent assessment where it is analyzed separately, with consistent scoring metrics applied to ensure an unbiased and objective confidence rating. The confidence score is determined using a precise 0-100 integer scale, which takes into account several key factors, including algorithmic efficiency, code complexity, and potential optimization opportunities. The plan with the highest score then proceeds to the next agent.

3.2.5 Test generation agent

Operational framework

The *Test Generation Agent* specializes in creating test cases for solution implementation. It analyzes the problem description and solution plan with the highest score from the *Ranking Agent* to develop a robust set of test cases that validate functionality, handle edge cases, and ensure complete requirement coverage.

Analytical Processing and Structured output

The agent follows a systematic approach to test creation, maintaining consistent XML formatting while introducing specific tags for test cases and validation criteria. Each test is developed through careful consideration of requirements, edge cases, and expected behaviors.

3.2.6 Coding agent

Operational framework

The *Coding Agent* functions as an experienced programming contestant, operating through a well-defined system prompt (Appendix A) that establishes its primary responsibility: implementing precise algorithmic solutions based on (a) problem descriptions, (b) solution plans, and (c) unit tests. The agent is specifically designed to create a single self-contained function that solves the given problem while adhering to strict input-output specifications.

Analytical processing and Structured output

Similar to other agents in the system, *Coding Agent* employs a chain-of-thought approach encapsulated within `<thinking>` tags, followed by implementation details within `<code>` tags. This structured approach ensures systematic problem analysis before code generation. The

agent processes three key input components: (a) the simplified problem description, (b) a solution plan containing pseudocode, and (c) a set of unit tests that define expected behavior.

Component-wise breakdown

1. **Function Implementation (<code>):** Generates a single, self-contained `solve()` function that:
 - Accepts input as a string parameter
 - Processes the input according to specification
 - Implements the required algorithm
 - Produces output in the specified format

3.2.7 Test evaluation node

Operational framework

The *Test Evaluation Node* serves as a crucial workflow component that validates the implementation generated by both *Coding* and *Debugger Agent*. Unlike the previous LLM-based agents, this node operates as a pure computational unit, executing the generated code against both public and private test cases within a controlled environment. The node ensures safe execution through timeout mechanisms and comprehensive error handling.

The *Test Evaluation Node* maintains system stability through:

- Strict timeout enforcement
- Memory usage control
- Input/output stream management
- Comprehensive error capturing
- Structured result formatting

Core functionality and Output structure

The node processes code execution results through the `TestEvaluator` class, which manages test case execution and result collection. The output is structured using Pydantic models, specifically the `TestEvaluationResult` class (Appendix B), which contains detailed information about test execution results, compilation errors, and debugging information – if at least one test fails –dedicated to the next agent.

Component-wise breakdown

1. **Input processing**
 - The agent receives the code from the coding step.

- Public and private test cases are used for validation.
 - Each test case consists of an input string and its expected output.
2. **Execution environment**
- The code runs within an isolated environment to ensure safety.
 - Standard output is redirected to capture execution results.
 - A timeout of 3 seconds is enforced to prevent infinite loops.
 - Exceptions are handled to manage unexpected runtime scenarios.
3. **Test case evaluation**
- (a) **Compilation**
- The code is checked for syntax errors and proper structure.
 - Ensures the presence of the required 'solve' function.
 - Any compilation errors are captured for debugging.
- (b) **Runtime execution**
- Each test case is executed separately.
 - Input and output streams are managed throughout the process.
 - Timeout constraints are applied to avoid long execution times.
 - Runtime errors or exceptions are recorded.
- (c) **Result comparison**
- The output is normalized by stripping extra whitespace.
 - Exact matching is performed between the actual and expected outputs.
 - Handles minor formatting variations to ensure robustness.
 - Long outputs are formatted for better readability.
4. **Output structure**
- *Status*: A summary of execution results or any error messages.
 - *All tests passed*: A boolean flag indicating if all tests were successful.
 - *Compilation error*: Details of any issues encountered during compilation.
 - *Debug information*:
 - Code that was executed.
 - Type of error encountered (compilation or runtime).
 - Compiler error messages, if any.
 - Information on failed test cases, including:
 - * Input values used in the test.
 - * Expected output values.
 - * Actual output produced by the code.
 - * Any error messages that occurred during execution.

3.2.8 Debugging Agent

Operational framework

The *Debugging Agent* functions as an expert debugging system, operating through a carefully crafted system prompt (Appendix A) that establishes its role in analyzing and fixing code

that has failed to pass test cases. The agent employs a systematic approach to debugging, combining thorough analysis of failed tests with structured reflection and strategic problem-solving.

Analytical processing and Structured output

The agent processes four key input components: (a) the original solution plan, (b) the failing code implementation, (c) detailed information about failed tests, and (d) any compiler errors if present. Following a systematic five-step reflection process encapsulated in `<thinking>` tags, the agent produces a comprehensive debugging analysis and solution structured into distinct XML sections.

Component-Wise Breakdown

1. **Error Analysis (`<analysis>`)**: Summary of identified errors and their likely causes.
2. **Implementation Reflection (`<reflection>`)**: Detailed reflection on how the implementation aligns with the problem description and solution plan.
3. **Debugging Strategy (`<strategy>`)**: Outline of the approach to fix the identified issues.
4. **Code Correction (`<code>`)**: Implements the debugging fixes.

4. Technical details

4.1 Frameworks overview

The system architecture leverages two primary frameworks: LangChain and LangGraph, chosen for their complementary capabilities in building complex LLM-powered applications.

LangChain provides foundational components for structured interaction with language models. It enables chaining multiple operations, such as prompt templates, LLM invocation, and structured output parsing, offering a robust abstraction for these tasks.

Building upon LangChain’s functionality, LangGraph extends its capabilities to orchestrate multi-agent workflows through undirected cyclic graphs. This approach allows explicit representation of dependencies and conditional transitions between different agents, enabling graph-based workflow orchestration and providing a more complex structure for managing multi-agent systems.

4.2 Graph-based workflow orchestration

The graph-based architecture enables flexible workflow modification, clear dependency management, and robust error handling while maintaining scalability and maintainability of the system.

4.2.1 Workflow representation

The multi-agent system is modeled as an undirected cyclic graph $G = (V, E)$, where:

- $V = \{v_1, \dots, v_n\}$ represents the set of agent nodes.
- $E = \{e_1, \dots, e_m\}$ represents the undirected edges defining potential interactions between agents, where each edge $e_i = \{v_j, v_k\}$ represents a bidirectional interaction between agents v_j and v_k .

4.2.2 State management

The system implements a stateless execution model through a centralized state class (Appendix B), defined using Pydantic schemas. This approach ensures:

- **State immutability:** Each agent interaction produces a new state object rather than modifying an existing state, following functional programming principles.

- **Type safety:** Pydantic schemas enforce strict typing and validation of state attributes, reducing runtime errors and improving system reliability.
- **State transitions:** The workflow graph defines valid state transitions based on mutual interactions between connected agents as:

$$S_{t+1} = T(S_t, I_t)$$

where:

- S_t represents the system state at time t .
- I_t represents the set of interactions occurring at time t , where $I_t \subseteq E$.
- T represents the state transition function based on the current state and agent interactions.
- S_{t+1} represents the resulting state after agent interactions.
- **Validation flow:** The validation flow for state transitions is represented as:

$$S_{t+1} = V(T(S_t, I_t))$$

where:

- V represents the Pydantic validation function.
- T represents the state transition function.
- S_t represents the current state.
- I_t represents the set of interactions at time t .

5. Experimental evaluation

5.1 Experimental setup

The experimental evaluation of the system was conducted using the DeepMind CodeContests dataset, a comprehensive benchmark specifically designed for evaluating code generation systems. The dataset has been extensively used in evaluating state-of-the-art systems, including all mentioned above, enabling direct comparisons with existing approaches. The problems are sourced from a diverse range of competitive programming platforms, including CodeForces, CodeChef, Aizu, AtCoder, and HackerEarth, ensuring a broad spectrum of problem types and difficulty levels. Each problem includes both public and private test cases, allowing for thorough validation of generated solutions and assessment of generalisation capabilities.

5.1.1 Dataset preprocessing

For this experiment a single source of problems is chosen, CodeForces, and selected 6 distinct difficulty levels ranging from A (the easiest) to F (the most challenging). From each difficulty level, 20 problems were randomly chosen, resulting in a total of 120 problems. Each problem includes a title, problem text, difficulty level, public test cases for initial validation, and private test cases for more thorough testing.

5.1.2 Language model utilised

For the code generation tasks, the system employed OpenAI's GPT-4o, a model recognized for its advanced capabilities and efficiency.

- **Enhanced performance:** GPT-4o is optimized for both speed and accuracy, making it an ideal choice for competitive programming tasks.
- **API pricing:**
 - Text input: \$2.50 per 1M tokens
 - Text output: \$10.00 per 1M tokens

5.2 Evaluation metrics

To comprehensively assess our system's performance, we collected multiple metrics during runtime. The following metrics were chosen to provide a detailed view of both the effectiveness and efficiency of the proposed multi-agent system, enabling comparison with baseline approaches and insights into areas for potential improvement.

5.2.1 Solution quality metrics

The success rate (`is_solved`) serves as our primary indicator of system effectiveness, determining whether a solution passes all test cases. For each problem, we track the specific algorithm chosen (`solution_algorithm_name`), providing insights into the system’s problem-solving strategies.

5.2.2 Efficiency metrics

The efficiency of our system is measured through several key metrics. Following the established standard in code generation evaluation (Y. Li et al., 2022; Ridnik et al., 2024), we primarily track pass@k rates to assess solution quality, while also monitoring the number of debugging attempts (`k_tries`) required to reach a solution. These metrics help us understand the system’s convergence patterns. Computational performance is tracked through the total execution time (`exec_time_sec`) and the number of successful LLM API calls (`total_llm_calls`), providing insights into the system’s real-world applicability.

5.2.3 Resource utilisation metrics

Resource utilization is another crucial aspect we monitor. We track token consumption across different stages: input tokens for prompts (`total_input_tokens`), output tokens for generated solutions (`total_output_tokens`), and the total token usage (`total_tokens`). These metrics, combined with the total API costs in USD (`total_costs_usd`), help us assess the system’s economic viability and identify opportunities for optimization.

5.2.4 Testing coverage metrics

The quality of our solutions is further validated through comprehensive test coverage metrics. We track both the initial number of public test cases (`number_given_pub_tests`) and the total number of test cases including private ones (`total_number_test_cases`). For unsuccessful solutions, we record the number of failed test cases (`number_failed_test_cases`), providing valuable insights into areas needing improvement.

6. Results and discussion

This section presents a comprehensive analysis of the proposed multi-agent system’s performance, examining key metrics across different dimensions and discussing both achievements and limitations observed during our experimental evaluation.

6.1 Key findings

6.1.1 Pass rate distribution

Approach	Pass@1	Pass@k
Direct prompting (GPT-4)	12.10	18.80
DeepSeek-Coder-6.7B	6.46	15.25
MoTCoder	9.29	16.97
AlphaCode	-	16.00
Reflexion	15.00	-
AlphaCodium	-	29.00
MapCoder (SOTA)	28.50	35.20
This system	26.50	35.04

Table 6.1: Pass rates for different approaches (k-values indicated in note)

Note: (i) $k = 5$ for all methods unless stated otherwise. AlphaCode $k = 1000$, Reflexion does not provide any other k results, and this system uses $k = 4$.

Source: Results are sourced from their respective papers: DeepSeek-Coder from (Guo et al., 2024), MoTCoder from (J. Li et al., 2023), AlphaCode from (Y. Li et al., 2022), Reflexion from (Shinn et al., 2023), AlphaCodium from (Ridnik et al., 2024), and Direct prompting and MapCoder from (Md. Ashraful et al., 2024).

Our experimental results demonstrate varying degrees of success across different complexity levels. The system achieved a baseline **Pass@1 rate of 26.50%**, which improved to **35.04% for Pass@4**, indicating that additional attempts often led to successful solutions. This improvement of **8.54 percentage points** suggests that our iterative debugging approach contributes positively to problem-solving capabilities, though with diminishing returns after the first few attempts.

6.1.2 Comparison with Baselines

The proposed system demonstrates competitive performance compared to existing state-of-the-art baselines. It achieved a **Pass@1 rate of 26.50%**, slightly below MapCoder’s

28.50%, but still outperforms other models such as Reflexion (**15.00%**) and MoTCoder (**9.29%**). For **Pass@k** (**k=4**), the system reached **35.04%**, which closely matches MapCoder’s **35.20%**, but also outperforms other models like AlphaCodium (**Pass@k = 29.00%**) and DeepSeek-Coder (**Pass@k = 15.25%**).

6.1.3 Difficulty-stratified analysis

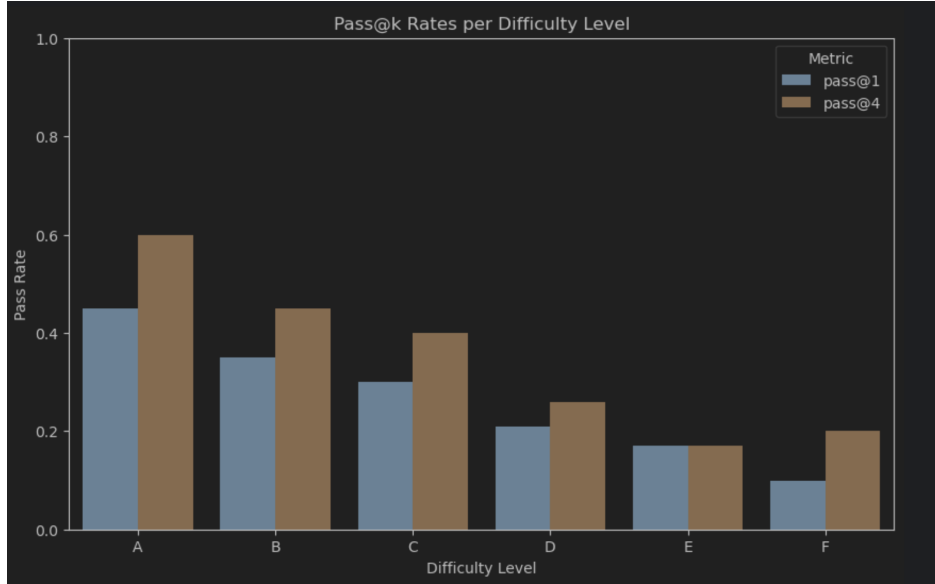


Figure 6.1: Pass rate distribution across difficulty levels

Level	Pass@1	Pass@4
A	45.00%	60.00%
B	35.00%	45.00%
C	30.00%	40.00%
D	21.00%	26.00%
E	17.00%	17.00%
F	10.00%	20.00%

Table 6.2: Pass rate distribution per problem difficulty level

The performance exhibits a clear correlation with problem difficulty. For Level A problems, the system achieved impressive Pass@1 and Pass@4 rates of **45.00%** and **60.00%** respectively, demonstrating strong capabilities in handling straightforward programming tasks.

However, these rates declined progressively through difficulty levels, reaching **10.00%** Pass@1 and **20.00%** Pass@4 for Level F problems, highlighting the challenges in solving more complex programming tasks.

6.1.4 Impact of self-correction mechanism

The improvement from **Pass@1** to **Pass@4** across difficulty levels demonstrates the system’s ability to refine solutions. For Level A, Pass@4 success increased by 15 percentage points. However, the minimal improvements in higher levels, such as Level F (10.00% to 20.00%), suggest limitations in resolving complex errors.

Analysis of the attempts distribution reveals that approximately **31 problems were solved on the first attempt**, with additional successes achieved in subsequent tries (**6 problems on the first debugging attempt**, and **4 problems after three debugging attempts**). This pattern suggests that the collaborative interaction between agents contributes to solution refinement, though with diminishing returns after the initial attempt.

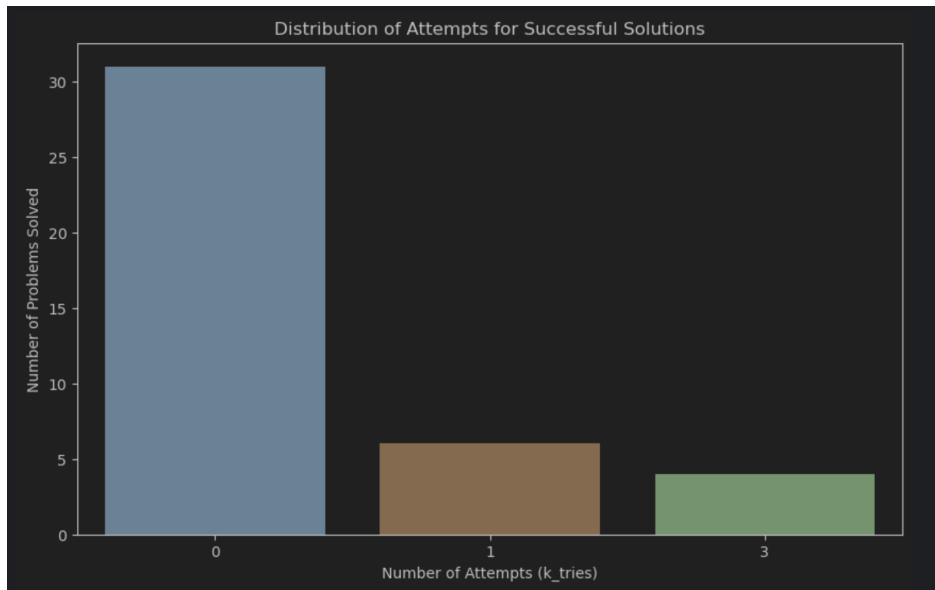


Figure 6.2: Distribution of debugging attempts

6.2 Testing and validation metrics

Our system undergoes rigorous validation against both public test cases (visible during development) and private test cases (hidden until submission), enabling comprehensive assessment of solution correctness and generalizability. For unsolved problems, our analysis reveals:

- Average **failed** tests rate: 62.63%
- Average **passed** tests rate: 37.37%

This partial success pattern yields two key insights: (a) the system demonstrates strong capability in grasping and implementing core algorithmic concepts, evidenced by the substantial percentage of passed tests even in unsuccessful attempts, and (b) the higher failure rate indicates specific challenges in handling edge cases, complex input scenarios, and performance

optimization requirements that prevent full solution acceptance. These validation results provide valuable guidance for future improvements in test coverage and solution optimization.

6.3 Resource utilisation analysis

6.3.1 LLM interaction patterns

The system's interaction with the LLM backend shows distinct patterns between successful and unsuccessful solutions:

- Average calls per **solved** problem: 8.39
- Average calls per **unsolved** problem: 11.00 (all possible steps)

The higher average call count for unsolved problems (31% more) suggests that additional LLM interactions don't necessarily translate to higher success rates, pointing to potential inefficiencies in the debugging process.

6.3.2 Computational efficiency

For successfully solved problems, the system demonstrated the following efficiency metrics:

- Average **solution time**: 151.29 seconds
- Average **USD cost** per problem: \$0.0883
- Average **token usage**: 17,082.46 tokens

These metrics provide a baseline for resource allocation and cost projections in practical applications of the system. While the average execution time of 151.29 seconds and cost of \$0.0883 per problem are acceptable for research purposes, the system's scalability remains a concern. Increased resource demands for complex problems may impact its applicability in production scenarios.

6.3.3 Comparison of calls for Solved problems with Baselines

6.4 Key patterns and implications

The comprehensive analysis of our results reveals several significant patterns:

1. **Complexity impact**: The clear degradation of success rates across difficulty levels (Figure 6.1) demonstrates that the system's effectiveness is strongly correlated with problem complexity. This relationship appears to be linear, with no particularly sharp drops in performance between levels.

Approach	LLM Calls	Tokens (k)	USD Costs
AlphaCode	1,000,000.00	-	-
AlphaCodium	20.00	-	-
MapCoder	19.00	38.70	~ \$0.2000
This system	8.39	17.08	\$0.0883

Table 6.3: Comparison of LLM calls, token usage, and costs for solved problems

Note: USD costs assuming the same LLM.

Source: Results are sourced from their respective papers: AlphaCode from (Y. Li et al., 2022), AlphaCodium from (Ridnik et al., 2024), and MapCoder from (Md. Ashraful et al., 2024).

2. **Initial solution quality:** The high concentration of successful solutions in the first attempt (as shown in Figure 6.2) suggests that the system’s initial solution generation is crucial. This is supported by the sharp decline in successful solutions in subsequent attempts.
3. **Testing coverage:** The significant proportion of passed tests even in unsolved problems (37.37%) indicates that the system often achieves partial success, suggesting potential value in solutions that don’t meet all test criteria.

6.5 Limitations and bottlenecks

Several key limitations emerge from our analysis:

1. **Scalability challenges:** The system’s performance characteristics raise important scalability considerations. While the average execution time of 151.29 seconds per solved problem is acceptable for research purposes, it may present challenges in production environments requiring rapid solution generation.
2. **Complexity barrier:** The significant performance drop between lower and higher difficulty levels (A-F) suggests a current ceiling in the system’s problem-solving capabilities.

6.6 Future work

Our analysis reveals both immediate opportunities for enhancement and exciting possibilities for revolutionary approaches to multi-agent systems.

6.6.1 Performance improvements

The current average cost of \$0.0883 and average execution time of 151.29 seconds per problem suggest room for optimization in both model selection and framework architecture.

While LangChain provides convenient abstractions for agent interactions, its multiple abstraction layers can introduce performance overhead due to synchronous execution patterns and additional memory management. Direct implementation of asynchronous components in pure Python could potentially reduce our average execution time by eliminating these intermediary layers. For instance, using Python’s native `asyncio` with direct API calls could minimize latency between agent communications and reduce memory overhead.

Frameworks like AutoGen offer a middle ground, providing structured agent communication while maintaining lower-level performance through its lighter abstraction layer. Meanwhile, LlamaIndex’s specialized data structures could optimize context management without the full overhead of LangChain. A hybrid approach combining direct asynchronous Python implementations for performance-critical components with specialized frameworks could provide the optimal balance of development efficiency and runtime performance.

This approach could involve a tiered architecture where different agents leverage different-sized LLMs based on task complexity, all connected through a lean, async-first implementation that minimizes abstraction overhead while maintaining system modularity.

6.6.2 Implement long-term memory

Instead of treating each problem independently, future iterations could implement a *long-term memory unit* where solutions and patterns from previously solved problems are saved to inform new approaches. By learning from successful solutions, this approach could potentially improve the Pass@1 rate beyond the current 26.50% and boost the Pass@4 rate above 35.04%, while simultaneously reducing the average number of LLM calls needed per solution.

6.6.3 Adaptive agent specialisation

Rather than maintaining fixed agent roles, a dynamic system could be explored where agents evolve and specialise based on problem patterns. This could involve agents that automatically identify their strengths (e.g., some becoming *optimisation experts* or *edge case specialists*) through continuous self-assessment of their performance across different problem types.

7. Conclusions

The primary objective of this thesis was to develop and evaluate a multi-agent system for solving complex algorithmic problems, particularly in competitive programming. The proposed architecture integrates specialised agents for tasks such as problem simplification, planning, test case generation, coding, and debugging, leveraging advanced prompting techniques like chain-of-thought reasoning.

The system was tested on the DeepMind CodeContests dataset, achieving a Pass@1 rate of **26.50%** and a Pass@4 rate of **35.04%**, demonstrating competitive performance. For simpler problems (Level A and B), the system excelled with Pass@4 rates of **60.00%** and **45.00%**, while performance for the most complex problems (Level F) remained limited at **20.00%**.

In terms of efficiency, the proposed architecture outperformed the state-of-the-art MapCoder system, achieving approximately 2.2 times lower USD costs and token usage, with an average cost per problem of **\$0.0883** and a token usage of **17,082.46 tokens per solution**. These results highlight the cost-effectiveness and resource efficiency of the system compared to existing approaches.

The goals of this thesis are considered fulfilled. Future work could focus on enhancing scalability, implementing memory-based learning for pattern recognition, and enabling adaptive agent behaviors to handle a broader range of problem complexities.

List of references

- Ahmad, W. U., Chakraborty, S., Ray, B., & Chang, K.-W. (2021). Unified pre-training for program understanding and generation. *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, 2655–2668. <https://doi.org/10.18653/v1/2021.naacl-main.211>
- Austin, J., et al. (2021). Program synthesis with large language models. *arXiv preprint, arXiv:2108.07732*. <https://arxiv.org/abs/2108.07732>
- Chen, M., et al. (2021). Evaluating large language models trained on code. *arXiv preprint, arXiv:2107.03374*. <https://arxiv.org/abs/2107.03374>
- Chowdhery, A., et al. (2022). Palm: Scaling language modeling with pathways. *arXiv preprint, arXiv:2204.02311*. <https://arxiv.org/abs/2204.02311>
- Feng, Z., Guo, D., Tang, D., Duan, N., Feng, X., Gong, M., Shou, L., Qin, B., Liu, T., Jiang, D., & Zhou, M. (2020). Codebert: A pre-trained model for programming and natural languages. *Findings of the Association for Computational Linguistics: EMNLP 2020*, 1536–1547. <https://doi.org/10.18653/v1/2020.findings-emnlp.139>
- Gulwani, S., Polozov, O., & Singh, R. (2017). Program synthesis. *Foundations and Trends® in Programming Languages*, 4(1-2), 1–119. <https://doi.org/10.1561/25000000010>
- Guo, D., Zhu, Q., Yang, D., Xie, Z., Dong, K., Zhang, W., Chen, G., Bi, X., Wu, Y., Li, Y. K., Luo, F., Xiong, Y., & Liang, W. (2024). Deepseek-coder: When the large language model meets programming – the rise of code intelligence [Version 2]. *arXiv preprint, arXiv:2401.14196*. <https://arxiv.org/abs/2401.14196>
- Knuth, D. E. (1992). *Literate programming* (Vol. 27). Center for the Study of Language; Information. <https://www-cs-faculty.stanford.edu/~knuth/lp.html>
- Li, J., Chen, P., Xia, B., Xu, H., & Jia, J. (2023). Motcoder: Elevating large language models with modular of thought for challenging programming tasks [Last revised 22 Aug 2024]. *arXiv preprint, arXiv:2312.15960v3*. <https://arxiv.org/abs/2312.15960>
- Li, Y., Choi, D., Chung, J., Kushman, N., Schrittwieser, J., Leblond, R., Eccles, T., Keeling, J., Gimeno, F., Dal Lago, A., et al. (2022). Competition-level code generation with alphacode. *Science*, 378(6624), 1092–1097. <https://doi.org/10.1126/science.abq1158>
- Md. Ashraful, I., et al. (2024). Mapcoder: Multi-agent code generation for competitive problem solving. *arXiv preprint, arXiv:2405.11403*. <https://arxiv.org/abs/2405.11403>
- Nijkamp, E., et al. (2022). A conversational paradigm for program synthesis. *arXiv preprint, arXiv:2203.13474*. <https://arxiv.org/abs/2203.13474>
- Parvez, M. R., Chakraborty, S., Ray, B., & Chang, K.-W. (2018). Building language models for text with named entities. *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2373–2383. <https://doi.org/10.18653/v1/P18-1221>
- Ridnik, T., et al. (2024). Alphacodium: A multi-agent system for code generation. *arXiv preprint, arXiv:2405.11403*. <https://arxiv.org/abs/2405.11403>

- Roziere, B., Lachaux, M.-A., Banville, H., Lhoest, Q., Gugger, S., Lambert, N., Goyal, N., Ott, M., & Lample, G. (2023). Code llama: Open foundation models for code. *arXiv preprint, arXiv:2308.12950*. <https://arxiv.org/abs/2308.12950>
- Shinn, N., et al. (2023). Reflexion: Language agents with verbal reinforcement learning. *arXiv preprint, arXiv:2303.03004*. <https://arxiv.org/abs/2303.11366>
- Sutskever, I., Vinyals, O., & Le, Q. V. (2014). Sequence to sequence learning with neural networks. *Advances in Neural Information Processing Systems*, 27, 3104–3112. <https://arxiv.org/abs/1409.3215>
- Touvron, H., Martin, L., Stone, K., Albert, P., Almahairi, A., Couairon, R., Thibaut, M., Sablayrolles, A., Lhoest, Q., Gugger, S., et al. (2023). Llama: Open and efficient foundation language models. *arXiv preprint, arXiv:2302.13971*. <https://arxiv.org/abs/2302.13971>
- Vaswani, A., et al. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30, 5998–6008. <https://arxiv.org/abs/1706.03762>
- Wang, Y., Wang, W., Joty, S., & Hoi, S. C. H. (2021). Codet5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation. *arXiv preprint, arXiv:2109.00859*. <https://arxiv.org/abs/2109.00859>
- Wei, J., et al. (2022). Chain of thought prompting elicits reasoning in large language models. *arXiv preprint, arXiv:2201.11903*. <https://arxiv.org/abs/2201.11903>
- Yao, S., et al. (2023). Tree of thoughts: Deliberate problem solving with large language models. *arXiv preprint, arXiv:2305.10601*. <https://arxiv.org/abs/2305.10601>

Appendices

A. System prompts

Simplifier agent

```
SYSTEM_PROMPT_TEMPLATE: Final[str] = """\
```

```
You are a world-class competitive programming problems solver.
```

```
Describe the given [Complex coding contest problem] in
```

```
bullet points, while addressing the problem context, objective,
```

```
inputs, outputs, rules, constraints, and other
```

```
relevant details that appear in the problem, excluding example explanation.
```

```
-----
```

```
Important: Do not solve the problem or explain examples. Focus on providing  
a clear and comprehensive description
```

```
of the problem itself. Before providing the description, carefully analyze  
the problem and think through its various
```

```
aspects. Think through each field step-by-step before writing it. The output  
should be correctly formatted as a XML
```

```
instance that conforms to the given schema below:
```

```
<title> # Problem title </title>
```

```
<thinking>
```

```
# Consider the real-world scenario or abstract setting of the problem
```

```
# Think about why this problem might be relevant or interesting
```

```
# Reflect on any domain-specific knowledge that might be helpful
```

```
</thinking>
```

```
<context> # Bullet points describing the problem context </context>
```

```
<thinking>
```

```
# Clearly identify the main goal of the problem
```

```
# Consider any secondary objectives or implicit goals
```

```
# Think about how the objective relates to the problem context
```

```
</thinking>
```

```
<objective> # Bullet points describing the problem objective </objective>
```

```
<thinking>
```

```
# Analyze the input format and types
```

```
# Consider the range and constraints of each input
# Think about any implicit information provided in the input
</thinking>

<inputs> # Bullet points describing the inputs </inputs>

<thinking>
# Identify the required output format
# Consider any specific formatting or precision requirements
# Think about how the output relates to the problem objective
</thinking>

<outputs> # Bullet points describing the outputs </outputs>

<thinking>
# Identify explicit rules stated in the problem
# Consider implicit rules that might be inferred
# Think about any edge cases or special conditions
# Avoid using unicode symbols, use words instead
</thinking>

<rules> # Bullet points describing rules </rules>

<thinking>
# Identify explicit rules stated in the problem
# Consider implicit rules that might be inferred
# Avoid using unicode symbols, use words instead
</thinking>

<constraints> # Bullet points describing constraints </constraints>

<thinking>
# Consider any other relevant information provided in the problem statement
# Think about any clarifications or explanations that might be helpful
# Reflect on how these details might impact the problem-solving approach
</thinking>

<relevant_details> # Bullet points describing any additional relevant details
</relevant_details>
"""

HUMAN_PROMPT_TEMPLATE = """\
Complex coding contest problem:
```

```
{original_problem}
```

Test cases:

```
{public_tests}
```

```
"""
```

Self-retrieval agent

```
SYSTEM_PROMPT_TEMPLATE: Final[str] = """\
```

```
Given the problem description, provide relevant problems then identify the  
algorithm behind it and also explain the tutorial of the algorithm.
```

```
# Exemplars:
```

```
Recall {t} relevant and distinct problems (different from problem mentioned above).|
```

```
For each problem:
```

1. Think about how it relates to the main problem and why it's relevant
2. Consider different approaches to solving the problem
3. Plan out the solution step-by-step as numbered list
4. Think through the code implementation in {programming_language}
5. Reflect on the underlying algorithm and how to explain it effectively

```
-----
```

```
Important: Before providing the solutions, think through the problem carefully and|  
consider various approaches. The output should be correctly formatted as the given|  
schema below:
```

```
<thinking>
```

```
# Analyze the main problem and consider its key characteristics
```

```
# Reflect on similar problems you've encountered and their relevance
```

```
# Consider what makes a problem distinct yet related to the main problem
```

```
</thinking>
```

```
<problem>
```

```
<description> # Describe the problem. </description>
```

```
<thinking>
```

```
    # Break down the problem into smaller steps as numbered list
```

```
    # Consider different strategies and their pros/cons
```

```
    # Think about how this problem relates to the main problem
```

```
</thinking>
```

```
<plan> # The step-by-step plan to solve the problem as numbered list. </plan>|
```

```
<thinking>
  # Consider how to implement the solution in {programming_language}
  # Think about potential edge cases or challenges in the implementation
</thinking>

<code> # Code to solve this problem in {programming_language} programming language.
</code>

<thinking>
  # Reflect on the underlying algorithm used in the solution
  # Consider how to explain this algorithm in a clear and concise manner
  # Think about common applications and variations of this algorithm
</thinking>

<algorithm>
  <name> # Identify the algorithm (Brute-force, Dynamic Programming,
  Divide-and-conquer, Greedy, Backtracking, Recursive, Binary search, and so on)
  that needs to be used to solve current problem. </name>
  <tutorial> # Write a useful tutorial about the above mentioned algorithms.
  Provide a high level generic tutorial for solving this types of problem.
  Do not generate code. </tutorial>
</algorithm>
</problem>

# similarly add 2 more problems here
<problem></problem>
<problem></problem>
"""

HUMAN_PROMPT_TEMPLATE = """\
# Problem description:
{simplified_problem}
"""
```

Planning agent

```
SYSTEM_PROMPT_TEMPLATE: Final[str] = """
Given examples of similar problem solutions, generate 3 distinct concrete
step-by-step algorithms to solve the given problem. Before providing the
algorithms, carefully analyze the problem, examples, and test cases to
develop a comprehensive understanding.
```

Important: Think through each algorithm step-by-step before writing it.
Clearly break down the logic and approach for each distinct solution.
The output should be correctly formatted as the given schema below:

<thinking>

- # Analyze the main problem and its requirements
- # Consider the similarities and differences with the example problems
- # Reflect on the test cases and what they reveal about edge cases or specific requirements
- # Brainstorm potential approaches, considering their pros and cons

</thinking>

<algorithm_name> # Identify the algorithm (Brute-force, Dynamic Programming, Divide-and-conquer, Greedy, Backtracking, Recursive, Binary search, and so on) that needs to be used to solve the given problem.

</algorithm_name>

<plan> # Concrete plan to implement the first algorithm as numbered list.

</plan>

<thinking>

- # Reflect on the first algorithm and consider how to create a distinct second approach
- # Think about different paradigms or techniques that could be applied
- # Consider trade-offs between time complexity, space complexity, and implementation simplicity

</thinking>

<algorithm_name> # Identify the algorithm (Brute-force, Dynamic Programming, Divide-and-conquer, Greedy, Backtracking, Recursive, Binary search, and so on) that needs to be used to solve the given problem. </algorithm_name>

<plan> # Concrete plan to implement the second algorithm as numbered list.

</plan>

<thinking>

- # Analyze the previous two approaches and brainstorm a third distinct strategy
- # Consider any aspects of the problem not fully addressed by the first two approaches
- # Think about innovative or unconventional methods that could be applied

</thinking>

<algorithm_name> # Identify the algorithm (Brute-force, Dynamic Programming, Divide-and-conquer, Greedy, Backtracking, Recursive, Binary search, and so on) that needs to be used to solve the given problem. </algorithm_name>


```
<plan> # Concrete plan to implement the third algorithm as numbered list.
</plan>
"""

HUMAN_PROMPT_TEMPLATE = """\
# Examples of similar problem solution:
<examples>
{example_problems}
</examples>

# Problem to be solved:
<problem>
{simplified_problem}

Test cases:
{public_tests}
</problem>
"""
```

Ranking agent

```
SYSTEM_PROMPT_TEMPLATE: Final[str] = """\
Given the competitive programming problem and the
plan to solve the problem in {programming_language}
tell whether the plan is correct to solve this problem.

-----

Important: The output should be correctly formatted as
a XML instance that conforms to the given schema below:

<plan> {gen_plan} </plan>

<thinking> # Discuss whether the given competitive
programming problem is solvable by using the given planning.
</thinking>

<confidence> # Confidence score regarding the resolvability
of the problem. Must be an integer between 0 and 100.
</confidence>

"""
```

```
HUMAN_PROMPT_TEMPLATE = """\n# Problem:\n{simplified_problem}\n\n# Planning:\n{gen_plan}\n"""
```

Test generation agent

```
SYSTEM_PROMPT_TEMPLATE: Final[str] = """\nYou are an expert software engineer tasked with creating comprehensive unit tests for a given problem before implementing the solution. Your goal is to generate a set of unit tests that will guide the development process and ensure the final implementation meets all requirements.
```

```
## Input Variables:\n- <PROBLEM_DESCRIPTION>: A detailed description of the problem to be solved.\n- <SOLUTION_PLAN>: An outline of the proposed solution approach.\n- <PUBLIC_TESTS>: Any pre-existing public tests that must be passed.
```

```
## Your Tasks:\n1. Analyze the <PROBLEM_DESCRIPTION> and <SOLUTION_PLAN> carefully.\n2. Identify key functionalities and edge cases that need to be tested.\n3. Create a comprehensive set of unit tests that cover:\n    - Basic functionality\n    - Edge cases\n    - Input validation\n    - Expected outputs\n    - Error handling\n4. Ensure your tests cover all aspects mentioned in the <SOLUTION_PLAN>.\n5. Incorporate any <PUBLIC_TESTS> provided into your test suite.\n6. For each test, provide:\n    - A detailed thinking process explaining how you arrived
```

at the test case within <thinking> tags

- A descriptive name
- Input values
- Expected output or behavior
- A brief explanation of what the test is checking

7. Use a consistent format for presenting the tests, preferably in a way that can be easily translated into actual unit test code.

Remember, these tests will be used to drive the development of the actual solution, so they should be thorough and \ cover all possible scenarios outlined in the <PROBLEM_DESCRIPTION> and <SOLUTION_PLAN>.

Output Format:

Present your tests in a clear, organized manner. Use markdown formatting for readability. For example:

```
'''markdown
```

```
<thinking>
```

```
# Before writing actual test cases, thoroughly consider each  
potential test case type, including edge cases and its  
necessary variations.
```

```
</thinking>
```

```
### Test 1: [Test Name]
```

```
<thinking>
```

```
# Consider the problem requirements  
# Analyze the solution plan  
# Formulate a clear description of the test's purpose  
# Determine critical functionality to test  
# Design input to test this functionality  
# Identify potential edge cases  
# Predict expected output based on correct implementation
```

```
</thinking>
```

```
- Input: [Specify input]
```

```
- Expected output: [Specify expected output]
```

```
- Explanation: [Brief explanation of what the test is checking]
```

```
### Test 2: [Test Name]
```

```
...
```

```
'''
```

```
"""
```

```
HUMAN_PROMPT_TEMPLATE: Final[str] = """\
```

```
1. PROBLEM_DESCRIPTION:
```

```
<PROBLEM_DESCRIPTION>
```

```
{PROBLEM_DESCRIPTION}
```

```
</PROBLEM_DESCRIPTION>
```

```
2. SOLUTION_PLAN:
```

```
<SOLUTION_PLAN>
```

```
{SOLUTION_PLAN}
```

```
</SOLUTION_PLAN>
```

```
3. PUBLIC_TESTS:
```

```
<PUBLIC_TESTS>
```

```
{PUBLIC_TESTS}
```

```
</PUBLIC_TESTS>
```

Now, generate a comprehensive set of unit tests based on the provided <PROBLEM_DESCRIPTION>, <SOLUTION_PLAN>, and <PUBLIC_TESTS>. Use the detailed thinking process to explain your reasoning for each test. Also include the given <PUBLIC_TESTS>.

```
"""
```

Coding agent

```
SYSTEM_PROMPT_TEMPLATE: Final[str] = """\
```

You are an experienced programming contestant tasked with implementing a single function that solves the given problem based on the unit tests and the solution plan.

```
## Input Components:
```

- <PROBLEM>: Simplified problem description
- <SOLUTION_PLAN>: Pseudocode describing the algorithmic approach
- <UNIT_TESTS>: Test cases with input parameters and expected outputs

```
## Implementation Requirements:
```

Create exactly ONE function 'solve' that:

- Takes input parameters as string, avoid requirement of manual input with input()
- Implements the algorithm described in the solution plan

- Prints the result (print or use I/O)

Output Format:

Present your solution in a clear, organized manner. Provide explanations before code sections within <thinking> tags, write code within <code> tags then. Follow the example format:

<thinking>

```
# Analyze relevant tests
# Consider edge cases from tests
# Describe the main purpose of this component
# Decide on necessary data structures
# Think about efficiency and potential optimizations
</thinking>
```

<code>

Example code

```
def solve(input_str: str) -> str:
    data = input_str.splitlines()
    T = int(data[0])
    results = []

    index = 1
    for _ in range(T):
        N, K = map(int, data[index].split())
        if N == 0:
            results.append(0)
            index += 1
            continue
        arr = list(map(int, data[index + 1].split()))

        dp = [[False] * (K + 1) for _ in range(N + 1)]
        dp[0][0] = True # 0 sum is always possible with 0
        elements

        for i in range(1, N + 1):
            dp[i][0] = True # 0 sum is possible with any number
            of elements
            for j in range(1, K + 1):
                if arr[i - 1] <= j:
                    dp[i][j] = dp[i - 1][j] or dp[i - 1][j - arr[i - 1]]
                else:
                    dp[i][j] = dp[i - 1][j]
```

```
        results.append(1 if dp[N][K] else 0)
        index += 2

    print('\n'.join(map(str, results)))
</code>

## Important Notes:
- Generate one function 'solve' that takes 'input_str' as input
  within <code> tags, without any comments or test cases
- All necessary processing must happen within the function
"""

HUMAN_PROMPT_TEMPLATE: Final[str] = """\
1. PROBLEM:
<PROBLEM>
{PROBLEM}
</PROBLEM>

2. SOLUTION_PLAN:
<SOLUTION_PLAN>
{SOLUTION_PLAN}
</SOLUTION_PLAN>

3. UNIT_TESTS:
<UNIT_TESTS>
{AI_GEN_TESTS}
</UNIT_TESTS>

Now, implement the solution in {PROGRAMMING_LANGUAGE} based
on the provided <PROBLEM>, <SOLUTION_PLAN> and <UNIT_TESTS>.
"""
```

Debugger agent

```
SYSTEM_PROMPT_TEMPLATE: Final[str] = """\
You are an expert debugging agent tasked with analyzing and
fixing code that has failed to pass public and private tests
in a test-driven development environment. Your goal is to
identify issues, reflect on the problems, and provide debugged
code that passes all tests.

## Input Variables:
```

- <SOLUTION_PLAN>: The original plan for implementing the solution.
- <ORIGINAL_CODE>: The code that failed to pass the tests.
- <FAILED_TESTS>: Detailed information about the failed tests, including test cases and expected outputs.
- <COMPILER_ERROR>: Any compiler or syntax errors encountered (if applicable).

Your Tasks:

1. Analyze the provided information carefully, focusing on:
 - Given solution plan
 - Compiler errors (if any)
 - Failed test cases
 - Discrepancies between expected and actual outputs
2. Engage in a comprehensive reflection process following these 5 steps:
<thinking>
 1. Identify the type of error (syntax, logical, runtime, etc.)
 2. Trace the code execution for failed test cases
 3. Compare the implementation against the solution plan
 4. Consider potential edge cases or scenarios not adequately handled
 5. Hypothesize about the root cause of the failures</thinking>
3. Develop a debugging strategy based on your reflection.
4. Implement fixes to address the identified issues:
 - Correct any syntax errors
 - Modify logic to handle failed test cases
 - Refactor code if necessary to better align with the problem description and solution plan, \avoid using of manual input with input()

Output Format:

Present your analysis, reflection, debugging strategy and debugged code solution within corresponding tags. \

Write debugged code within <code> tags, do not include “” blocks.

Follow the example format:

1. Error Analysis

<analysis>

[Summary of identified errors and their likely causes]

</analysis>

2. Reflection on Implementation

<reflection>

```
[Detailed reflection on how the implementation aligns with the
problem description and solution plan]
</reflection>
```

3. Debugging Strategy

```
<strategy>
[Outline of the approach to fix the identified issues]
</strategy>
```

4. Debugged Code

```
<code>
[Your debugged code implementation]
</code>
"""
```

```
HUMAN_PROMPT_TEMPLATE = """\
```

```
1. SOLUTION_PLAN:
```

```
<SOLUTION_PLAN>
{SOLUTION_PLAN}
</SOLUTION_PLAN>
```

```
2. ORIGINAL_CODE:
```

```
<ORIGINAL_CODE>
{ORIGINAL_CODE}
</ORIGINAL_CODE>
```

```
3. FAILED_TESTS:
```

```
<FAILED_TESTS>
{FAILED_TESTS}
</FAILED_TESTS>
```

```
4. COMPILER_ERROR (if any):
```

```
<COMPILER_ERROR>
{COMPILER_ERROR}
</COMPILER_ERROR>
```

```
Now, analyze the provided <SOLUTION_PLAN>, <ORIGINAL_CODE>,
<FAILED_TESTS>, and <COMPILER_ERROR> (if any).
```

```
"""
```


B. State manager schema

Source code 1: State class using TypedDict

```
1 import operator
2 from typing import (
3     TypedDict, List, Optional, Annotated, Union, Literal
4 )
5 from pydantic import BaseModel, Field, conlist
6
7 class State(TypedDict):
8     """
9     Represents the complete state of the problem-solving workflow.
10
11     Input States:
12     - Initialized at the start of the workflow
13
14     Workflow States:
15     - Updated during the execution of the workflow
16     """
17
18     # Input states - Required at initialization
19     original_problem: str
20     public_tests: List[TestCase]
21     private_tests: List[TestCase]
22     programming_language: str
23     k_debug: int
24     t_plan: int
25
26     # Workflow states - Updated during execution
27     k_tries: Optional[int]
28     k_current: Optional[int]
29     t_current: Optional[int]
30     simplified_problem: Optional[SimplifiedProblemState]
31     example_problems: Optional[ProblemSetState]
32     gen_plans: Optional[Union[PlanningState, RankingState]]
33     current_plan: Optional[Union[PlanningState, RankingState]]
34     ai_gen_tests: Optional[TestCase]
35     code: Optional[str]
36     test_evaluation_result: Optional[TestEvaluationResult]
```

C. Sampled problem results

The source code, including sampled problem results, can be found on the author's GitHub page by the following URL: <https://github.com/MaksimObukhov/self-corrective-coding-agent/>